

Log Anomaly Detection with Hawkes-Process Timing and LLM Triage on System Logs

A detailed, self-contained, and honestly-validated study on public log data

Internship project report

July 16, 2026

Abstract

This report documents an end-to-end anomaly-detection system for system logs, built entirely on public data (the `loghub` HDFS and BGL datasets). The application domain is cybersecurity operations, but the primary objective is a technically strong, rigorously validated study of *event / time-series* data. The system has three parts. **(i)** An unsupervised **autoencoder detector**: a neural network is trained only on *normal* log activity, and the error it makes when reconstructing a new window of activity is used as an anomaly score; an Isolation Forest is the baseline it must beat. **(ii)** A **univariate Hawkes point process**, the mathematical core: log events cluster in time (one failure triggers cascades of related events), and a Hawkes process is the canonical model of such *self-excitation*. We fit it by maximum likelihood, estimate its *branching ratio* (a single interpretable number for how cascade-prone the stream is), validate the fit with the *time-rescaling theorem*, and propose its conditional intensity as an extra detector feature. **(iii)** An **LLM triage agent** that deduplicates raw alerts, correlates them into incidents (using the same self-excitation idea), and emits ranked, structured, human-readable incident reports. Validation is deliberately strict throughout: a never-shuffled temporal train/test split, precision–recall AUC instead of accuracy, a headline metric of *false alarms per day at a fixed detection rate*, and an explicit check for performance *drift* over time. Because the report is written to be understood from first principles, every non-standard concept is introduced with intuition and a worked example before it is used. Equally deliberately, we report what worked *and what did not*: HDFS detection is a strong *reproduction* of well-known results (PR-AUC 0.987), not a novelty; the Hawkes branching ratio on BGL is near-critical ($\eta \approx 0.97\text{--}1.0$), a genuine finding, but a single-exponential kernel underfits the deepest inter-event gaps; the timing feature roughly *halves* false alarms on BGL though absolute detection there is hard; and extreme-value thresholding—the principled tool—does not fit HDFS’s discrete scores, so a distribution-free alternative is used there. Treating these honest findings as first-class outcomes is itself a design choice.

Contents

1	Introduction: the problem, the goal, and how to read this report	3
1.1	The setting	3
1.2	The goal (and what it is not)	3
1.3	How to read this report	3
2	A primer on the building blocks	3
2.1	Logs, events, and templates	4
2.2	Supervised vs. unsupervised; why unsupervised here	4
2.3	Why accuracy is the wrong metric, and what to use instead	4
2.4	Temporal validation and the sin of leakage	5
2.5	The two detectors: Isolation Forest and autoencoder	5
2.6	Point processes: from Poisson to Hawkes	6
2.7	The time-rescaling theorem: how to check a point-process fit	6
2.8	Extreme-Value Theory: self-calibrating a threshold	7
3	Background and related work: prior art vs. our contribution	7
4	Data: two datasets, and why both are necessary	8

5	Log parsing with Drain	9
6	Exploratory analysis: the measurements that shaped the design	10
7	Detection on HDFS	10
7.1	The detection unit and the features	10
7.2	The validation protocol (the heart of the project)	11
7.3	The two models	11
7.4	Results	11
8	The mathematical core: Hawkes process on BGL	12
8.1	The model, restated with meaning	12
8.2	Estimating (μ, α, β) by maximum likelihood	13
8.3	Correctness before trust: recovery from simulation	13
8.4	Validating the fit with the time-rescaling theorem	13
8.5	Results and interpretation	13
9	Does the timing signal actually help detection?	15
10	Adaptive thresholding: stabilising false alarms under drift	16
10.1	The problem, made concrete	16
10.2	Extreme-Value Theory: the principled tool (derivation)	16
10.3	An honest negative finding, and the fix	17
10.4	Result	17
11	LLM triage: from raw alerts to ranked incidents	18
12	The dashboard and software architecture	18
13	Results at a glance	19
14	Honest limitations	20
15	Conclusion and future work	20
A	Notation	20
B	Glossary	21

1 Introduction: the problem, the goal, and how to read this report

1.1 The setting

Every server, database, and distributed system continuously writes *logs*: timestamped lines of text recording what it is doing. A moderately sized cluster produces millions of log lines per day. Buried in that torrent are the events an operator actually cares about—a disk failing, a node becoming unreachable, a brute-force login attempt, a cascade of retries triggered by one root cause. Finding those events is **log anomaly detection**. Doing it well is hard for three reasons that recur throughout this report:

1. **Anomalies are rare.** In our data they are 0.1–3% of all activity. A model that simply predicts “normal” every time is right $\sim 98\%$ of the time and completely useless. This forces us to measure success carefully (Section 2.3).
2. **Anomalies are unlabeled.** In the real world you do not have a labeled catalogue of every attack. So we train *unsupervised*: the model learns what *normal* looks like and flags departures from it. Labels are used *only* to grade the model afterwards, never to train it.
3. **Events cluster in time.** Logs are bursty: one root cause emits a cascade of related lines. This is not noise to be smoothed away—it is *structure*, and modeling it is the intellectual centerpiece of the project (Section 8).

1.2 The goal (and what it is not)

This is an internship project on a cybersecurity team, but the real objective is to build something *technically strong on time-series/event data, with rigorous validation*. Cybersecurity is the application; the transferable skill is disciplined stochastic modeling and honest measurement. Two consequences follow, and they shape everything:

- **Detection accuracy is not the contribution.** Detecting anomalies in the HDFS dataset with count features and an autoencoder is one of the most reproduced results in the entire log-analysis literature; dozens of public notebooks achieve what we achieve. We reproduce it *on purpose*, as a validated baseline and as a source of real anomalies for the triage layer—but we never pretend it is new. The un-commodity parts are the *timing model*, the *validation discipline*, the *drift-aware self-calibrating threshold*, and the *trriage layer*. Section 3 draws this line explicitly.
- **Honesty is a feature.** A rigorously-validated *negative* or *nuanced* result (“the timing feature helps, but only modestly, on a task that is genuinely hard”) is worth more than a suspiciously perfect number. Where the data contradicted a plan, we changed the plan and said so.

1.3 How to read this report

Section 2 is a self-contained primer on every concept the rest of the report uses—read it if any of *autoencoder*, *precision–recall*, *temporal leakage*, *point process*, *Hawkes process*, *branching ratio*, *time-rescaling*, *extreme-value theory* is unfamiliar; skip it otherwise. Section 3 separates prior art from our contribution. Sections 4–11 are the work itself, in pipeline order. Sections 13–15 summarise, list limitations honestly, and conclude. A notation table and glossary are in the appendix. Throughout, **blue boxes** give intuition and **green boxes** state the one thing to remember.

2 A primer on the building blocks

This section teaches, from first principles, the concepts used later. A reader fluent in machine learning and point processes can skip to Section 3.

2.1 Logs, events, and templates

A raw log line is unstructured text, e.g.

```
081109 203515 148 INFO dfs.DataNode: PacketResponder 1 for block blk_38865 terminating
```

To a computer this is just characters. But thousands of lines share the same *shape*, differing only in variable parts (the block number, an IP). A **log template** (or *event type*) is that shape with the variable parts blanked out:

```
PacketResponder <*> for block <*> terminating
```

Log parsing is the automatic discovery of these templates. After parsing, each raw line becomes a compact record: (timestamp, event_id, template, ...), and a stream of messy text becomes a stream of *categorical events*—something we can do statistics and machine learning on. We use **Drain** for this (Section 5).

2.2 Supervised vs. unsupervised; why unsupervised here

In *supervised* learning you show the model many labeled examples (“this is an attack, this is not”) and it learns the boundary. That requires a comprehensive labeled catalogue of anomalies, which nobody has—new failures and attacks appear constantly. In *unsupervised* anomaly detection you show the model only *normal* data, it builds a model of normality, and anything that does not fit is anomalous. This generalises to unseen anomaly types, which is exactly what operations needs.

Intuition

Learning only “normal” is like a bank teller who has only ever seen genuine banknotes. They can flag a forgery they have never seen before, because it fails to match everything genuine they know—without ever having been shown a catalogue of forgeries.

2.3 Why accuracy is the wrong metric, and what to use instead

Suppose 2% of items are anomalies. A lazy model that labels *everything* “normal” achieves 98% **accuracy**—and catches zero anomalies. So accuracy is worthless under class imbalance. We need metrics that focus on the rare positive class. Define, for a chosen alert threshold, the confusion counts:

	predicted anomaly	predicted normal
actually anomaly	TP (true positive)	FN (false negative)
actually normal	FP (false positive)	TN (true negative)

- **Recall** (a.k.a. detection rate) = $\frac{TP}{TP + FN}$: of all real anomalies, what fraction did we catch?
- **Precision** = $\frac{TP}{TP + FP}$: of everything we flagged, what fraction was a real anomaly? (High precision = few false alarms.)

A detector outputs a continuous *score*, not a yes/no; sweeping the threshold from strict to lenient traces out a curve of (recall, precision) pairs. The area under that curve is the **PR-AUC** (precision–recall area under curve, a.k.a. average precision). It summarises detection quality across all operating points in a single number that—unlike accuracy or the more common ROC-AUC—is honest under heavy imbalance.

Intuition

The “chance” PR-AUC of a coin flip equals the anomaly rate itself (e.g. 0.02). So a PR-AUC of 0.75 against a 0.02 baseline is $\sim 37\times$ better than random; a PR-AUC of 0.99 is near-perfect ranking. ROC-AUC, by contrast, can look flatteringly high (> 0.9) even for a weak detector when positives are rare—which is why we do not report it.

The operational metric: false alarms per day. PR-AUC grades the *ranking*. But an on-call analyst lives with a concrete number: at the detection rate we commit to (say “catch 90% of anomalies”), *how many false alarms will I get per day?* That is the headline metric. It is computed by fixing recall at 90%, reading off the resulting false-positive count, and dividing by the real elapsed time. A model with the same PR-AUC but fewer false alarms per day is strictly better in practice.

Key takeaway

Under class imbalance, report *PR-AUC* (ranking quality) and *false alarms per day at a fixed detection rate* (operational cost). Never accuracy.

2.4 Temporal validation and the sin of leakage

To estimate how a model will perform on the future, you must test it on data it did not train on. The naive way—shuffle all data and randomly split into train/test—is *wrong* for time series, because it lets the model peek at the future: a shuffled test set contains moments interleaved with training moments, so the model effectively trains on information from the test period. This is **data leakage**, and it inflates results.

The correct way for time series is a **temporal split**: train on the earlier portion of the stream, test on the strictly later portion, *never shuffled*. Nothing from the test period—not a label, not a scaling constant, not a threshold—may be computed using future data. This is stricter and yields lower, *honest* numbers.

Intuition

Predicting the future is the whole point. If you shuffle time, you are letting the student see next week’s exam questions while revising—the grade looks great and means nothing.

2.5 The two detectors: Isolation Forest and autoencoder

Isolation Forest (the baseline). Intuition: anomalies are “few and different,” so they are easy to *isolate*. The algorithm builds many random binary trees, each repeatedly splitting the data on a random feature at a random value. A normal point, surrounded by many similar points, needs many splits to be isolated into its own leaf; an anomaly, being far from the crowd, is isolated in just a few. The average number of splits needed becomes the anomaly score (fewer splits = more anomalous). It is fast, needs no labels, and is the standard baseline—so it is exactly what our neural detector must beat.

Autoencoder (the detector). An **autoencoder** is a neural network trained to copy its input to its output through a narrow *bottleneck* (Figure schematic: $48 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 48$). Because the middle layer is small, the network cannot memorise; it must learn the essential regularities of the data to reconstruct it. Train it *only on normal data* and it becomes an expert at reconstructing normal patterns. Feed it an anomaly—a pattern it never learned—and it reconstructs it poorly. The **reconstruction error** (mean squared difference between input and output) is therefore the anomaly score.

Intuition

A mechanic who has serviced only healthy engines can rebuild one blindfolded, but fumbles a strange fault they have never encountered. The size of the fumble measures how abnormal the engine is.

2.6 Point processes: from Poisson to Hawkes

A **point process** is a random model of *when* events happen on a timeline. Its central object is the **conditional intensity** $\lambda(t)$: the instantaneous rate of events at time t , given everything that happened before t . Loosely, $\lambda(t) dt$ is the probability of an event in the next instant dt .

The Poisson baseline (no memory). The simplest process has constant intensity $\lambda(t) = \mu$: events arrive at a steady average rate, and the past tells you nothing about the future. Real logs are emphatically *not* like this—they come in bursts.

The Hawkes process (events beget events). A **Hawkes process** adds *self-excitation*: every event temporarily raises the intensity, making further events more likely, and that boost decays over time. With an exponential decay kernel,

$$\lambda(t) = \underbrace{\mu}_{\text{background}} + \sum_{t_i < t} \underbrace{\alpha e^{-\beta(t-t_i)}}_{\text{echo of each past event}}, \quad (1)$$

where $\mu > 0$ is the baseline rate (events that arrive “on their own”), α is the jump in intensity caused by each event, and β controls how fast that jump fades. Each past event t_i adds a decaying bump; the intensity is the background plus the sum of all the bumps still echoing.

Intuition

Earthquakes are the textbook example: a main shock triggers aftershocks, each aftershock can trigger its own smaller ones, and the activity decays back to background. Logs behave the same way—one failure triggers retries, timeouts, and error cascades that fade once the incident resolves.

The branching ratio (one number that says it all). Integrate the echo of a single event over all future time: it triggers, on average,

$$\eta = \int_0^\infty \alpha e^{-\beta s} ds = \frac{\alpha}{\beta} \quad (2)$$

direct offspring events. This is the **branching ratio**. It is exactly the mean number of children in a Galton–Watson branching process, so the same dichotomy applies: if $\eta < 1$ (*subcritical*) cascades die out and the process is stable; if $\eta > 1$ (*supercritical*) they explode; $\eta = 1$ is the critical edge. A branching ratio of, say, 0.5 means each event spawns half an event on average—moderate clustering; 0.98 means the system is on the brink of self-sustaining cascades.

Key takeaway

The branching ratio $\eta = \alpha/\beta$ is a single, interpretable number for how cascade-prone a log stream is. Estimating it is a headline result of this project.

2.7 The time-rescaling theorem: how to check a point-process fit

How do you know a fitted intensity $\lambda(t)$ is any good? The **time-rescaling theorem** gives a clean test. Define the *compensator* $\Lambda(t) = \int_0^t \lambda(s) ds$: the total “expected number of events” the model

says should have occurred by time t . If you re-clock the timeline by Λ —running the clock fast when the model expects many events, slow when it expects few—then, *if the model is correct*, the events become a plain Poisson process of rate 1. Concretely, the rescaled gaps between consecutive events, $\Delta\tau_i = \Lambda(t_i) - \Lambda(t_{i-1})$, become independent $\text{Exp}(1)$ (unit-mean exponential) draws. So we fit the model, compute the $\Delta\tau_i$, and check they look $\text{Exp}(1)$ with a QQ-plot and a Kolmogorov–Smirnov test. Deviations reveal exactly where the model is wrong.

Intuition

It converts “is my timing model right?” into “do these transformed numbers look like a fair exponential die?”—a question with standard, visual answers. The branching ratio is meaningless if the fit fails this test, so it is the guardrail on the whole Hawkes analysis.

2.8 Extreme-Value Theory: self-calibrating a threshold

A detector outputs scores; a *threshold* converts scores to alerts. Picking a fixed threshold is fragile: if the score distribution drifts over time, a fixed line lets in a flood of false alarms (or misses everything). **Extreme-Value Theory** (EVT) offers a principled, self-calibrating alternative. Its key result (Pickands–Balkema–de Haan) is that the *tail* of almost any distribution—the exceedances above a high threshold—follows a **Generalized Pareto Distribution** (GPD), regardless of the underlying distribution’s shape. So we fit a GPD to recent high scores and invert it to find the exact level exceeded with a chosen small probability q (say “flag the top 0.1%”). As the data drifts we re-estimate the tail, and the threshold moves with it, holding the false-alarm rate steady. This is the **Peaks-Over-Threshold** (POT) method.

Intuition

Instead of nailing a flood-warning line at a fixed height, EVT models how high the river’s rare peaks get and sets the line to be crossed, say, once a century—and re-computes it as the river’s behaviour changes.

3 Background and related work: prior art vs. our contribution

Because “what is new here?” is the first question a professional reader asks, we answer it up front, component by component. Table 1 summarises; the prose gives context and citations.

Log parsing. Turning free text into templates is *solved*. Drain [1] is a fixed-depth parse-tree parser, the de-facto standard, shipped with benchmark configurations in the **logparser** toolkit. **Prior art.** **Ours:** we use it unchanged with the published configuration—parsing is infrastructure here, and the standard settings keep our templates comparable to the literature.

Log anomaly detection. A large literature. Classical methods build a per-session event-count matrix and apply PCA, Invariant Mining, or Isolation Forest (the **loglizer** benchmark [2]). Deep methods model event *sequences*: DeepLog [3] and LogAnomaly [4] use LSTMs to predict the next event and flag surprises; count-vector autoencoders are a standard unsupervised baseline. On HDFS these routinely reach F1/PR-AUC of 0.95–0.99, with dozens of public reproductions. **Prior art.** **Ours:** we reproduce the count-vector autoencoder (0.987) precisely because it is known-good—it validates the pipeline and supplies real anomalies to the triage layer. We do *not* claim it. Our detection-side contribution is the *validation protocol* and the *drift analysis* that most reproductions omit.

Point processes for logs. Hawkes processes [5] are standard in seismology, finance, and social media; Ogata’s thinning [6] is the standard simulator; the time-rescaling theorem [7] is the standard

goodness-of-fit tool. Their use as a *feature source for log anomaly detection* is uncommon—most log-anomaly work discards fine-grained timing. *Largely novel in this context.* **Ours:** we fit an exponential-kernel Hawkes process to BGL event times, estimate the branching ratio, validate with time-rescaling, and—the crucial step—*measure* whether the resulting self-excitation signal improves detection, reporting the answer honestly.

Adaptive thresholding. EVT/POT with the GPD is classical [8]; applying it to *streaming* anomaly scores is SPOT/DSPOT [8]. *Prior art.* **Ours:** we implement POT/GPD and a streaming variant, and we report a specific honest negative—EVT assumes a continuous tail, but the HDFS autoencoder scores are strongly discrete, so GPD extrapolation misbehaves there; we therefore use a distribution-free rolling quantile on HDFS and reserve EVT for BGL’s continuous scores. The drift-stabilisation *demonstration* is ours.

LLM triage. Using LLMs to summarise or triage alerts is recent and largely engineering-driven. *Emerging prior art.* **Ours:** the novel element is that *incident correlation is grounded in the self-excitation model*—a cascade of temporally clustered anomalies becomes one incident—rather than an ad-hoc window heuristic, with strictly-typed JSON output.

Table 1: Prior art vs. our contribution, by component.

Component	Already established (prior art)	What we did / contributed
Parsing	Drain is the standard parser (loghub).	Use it unchanged; standard config for comparability.
Detection	Count-vector autoencoder / IsoForest on HDFS is heavily reproduced (0.95–0.99).	Reproduce as a validated baseline; contribution is the strict protocol + drift analysis around it.
Timing (Hawkes)	Hawkes + time-rescaling standard elsewhere; rare for logs.	Fit on BGL, estimate branching ratio, validate, and <i>measure</i> whether timing aids detection.
Thresholding	EVT/POT and SPOT/DSPOT for streams.	Implement both; report EVT fails on discrete HDFS scores; demonstrate drift stabilisation.
Triage	LLMs for alert summarisation (recent).	Incident correlation grounded in self-excitation; typed JSON output.
Validation	Temporal splits and PR-AUC known in principle, often skipped in practice.	Applied rigorously end-to-end; FA/day headline; per-period drift.

4 Data: two datasets, and why both are necessary

We use *two* public loghub datasets. This was not a stylistic choice—it was forced by a measurement about timestamps, explained below.

Table 2: The two datasets and their roles.

	HDFS	BGL
Full name	Hadoop Distributed File System	Blue Gene/L supercomputer
Role in this project	Detection + triage showcase	Hawkes-process centerpiece
Raw size	11,175,629 lines	4,713,493 lines
Templates (Drain)	48	429
Time span	~1.6 days	~214.7 days
Labels	per <i>block</i> (2.93% anomalous)	per <i>line</i> (7.39% alerts)
Timestamp resolution	1 second	microsecond

What a “block” is (HDFS). HDFS stores files in fixed-size chunks called *blocks*, each identified by a `blk_...` id. A block’s life (write, replicate, serve, delete) generates many log lines over its short lifetime, and the ground-truth labels are provided *per block*: each `blk_...` is marked Normal or Anomaly. So on HDFS the natural unit of detection is the *block session*—all lines belonging to one block (Section 7).

What an “alert” is (BGL). BGL is a supercomputer’s RAS (Reliability, Availability, Serviceability) log. Each line carries a label field: `-` means non-alert, anything else is an alert category (e.g. a kernel fault). So BGL is labeled *per line*.

The decisive measurement: timestamp resolution. A continuous-time Hawkes process needs to know the *order* of events precisely—which event came first. We measured whether each dataset provides that ordering:

- **HDFS records time only to the second**, at ~80 events/second. The consequence, measured directly: **98.8%** of consecutive events share an identical timestamp (a “tie”). Even *within* a single block, 90.5% of consecutive events tie, and every block’s entire life spans ≤ 54 seconds (median 7s), occupying only ~2 distinct seconds. There is essentially no usable temporal ordering, at any level. One might “fix” this by adding random sub-second jitter—but that would *invent* the very sub-second structure the Hawkes process is supposed to measure, making the branching-ratio estimate an artifact of the jitter. We rejected it.
- **BGL records time to the microsecond** (e.g. 2005-06-03-15.42.50.675872), with 0% ties and events naturally spread over seconds. This is genuine continuous time—exactly what a Hawkes fit needs.

Key takeaway

HDFS’s 1-second timestamps make a continuous-time point process ill-posed, so HDFS hosts *detection* (its clean per-block labels are ideal for false-alarms-per-day). BGL’s microsecond timestamps make it the only place the Hawkes math is well-posed, so BGL hosts the *timing* work. Two independent benchmarks is a strength, not redundancy.

5 Log parsing with Drain

Drain builds a fixed-depth tree that routes each incoming line to a cluster of similar lines, creating a new template only when nothing matches closely enough. The depth and a similarity threshold are its hyperparameters; we use the published loghub values so our templates match the benchmark. After parsing, each line is (timestamp, event_id, template, ...); HDFS lines also carry their `block_id`, BGL lines their alert label and microsecond timestamp.

Result and a correctness check. The full HDFS log parses to **48** templates, BGL to **429**. We sanity-checked correctness with a coincidence that must hold if parsing is right: the HDFS template `BLOCK* NameSystem.allocateBlock` fires exactly once when a block is created, so its occurrence count should equal the number of blocks. It does—exactly 575,061—matching the label file to the line. (*Code: `src/logtrriage/parsing/`, `scripts/parse.py`.*)

6 Exploratory analysis: the measurements that shaped the design

Before modeling, we measured the data. Three findings changed the plan.

(1) Events cluster hard—self-excitation is real. The **Fano factor** (the variance of per-minute event counts divided by their mean) equals 1 exactly for a memoryless Poisson process; anything larger means the events are more bursty than random. We measured 5.3 on the HDFS sample and 698–2286 on BGL. This is the *empirical* justification for a Hawkes model, established before any fit. Figure 1 (left) shows it visually: long flat baselines interrupted by sharp vertical bursts.

(2) The HDFS timestamp problem (Section 4), which relocated the Hawkes work to BGL.

(3) Anomaly-rate drift. On the temporal split (Section 7), the earlier 70% of blocks are 3.26% anomalous and the later 30% are 2.16%. The base rate itself is *not stationary*—a first, concrete sign of the drift that later dominates the operational results. Figure 1 (right) shows the block-session size distribution (median 19 events/block); the long tail of unusually large blocks is disproportionately the anomalous ones (a failing block emits extra retry/error lines).

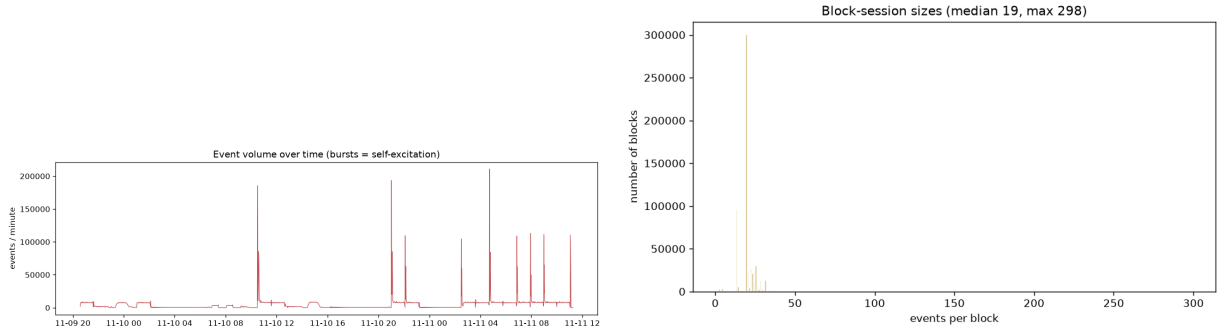


Figure 1: **Left:** HDFS event volume per minute—a flat baseline punctuated by sharp bursts, the visible signature of self-excitation (Fano factor $5.3 \gg 1$). **Right:** block-session sizes (median 19, max 298); the long tail is disproportionately the anomalous blocks.

7 Detection on HDFS

7.1 The detection unit and the features

Because HDFS labels are per block and a block lives ≤ 54 s, the unit of detection is the **block session**. We represent each block by a **count vector**: a 48-dimensional vector whose j -th entry is how many times template j occurred in that block (the standard “event count matrix”). This throws away timing (justified on HDFS, whose timing is unusable) and keeps *which* events occurred and *how often*—a compact fingerprint of the block’s behaviour. There are 575,061 such vectors.

7.2 The validation protocol (the heart of the project)

- **Strict temporal split.** Blocks are ordered by their start time; the earliest 70% (402,543 blocks) are training, the latest 30% (172,518 blocks) are test. The boundary falls at 2008-11-11 06:00:46. Nothing from the test period touches training, scaling, or thresholds (Section 2.4).
- **Train on normal only.** The autoencoder trains on the 389,428 *normal* training blocks; anomaly labels never enter training.
- **PR-AUC**, not accuracy or ROC (Section 2.3).
- **False alarms per day at 90% recall**, on real elapsed test time.
- **Drift:** every metric is also reported over consecutive chronological slices of the test window.

An honest subtlety we surface: splitting by block *count* (the loglizer convention) makes the test window time-dense—the 172,518 test blocks span only ~ 5 hours because blocks cluster near the end—so “per day” is computed from the *actual* ~ 5 -hour span, not assumed uniform.

7.3 The two models

- **Isolation Forest (baseline)**, fit on $\log(1 + \text{counts})$ (the log tames a few very frequent templates); anomaly score = $-\text{score_samples}$ (oriented so higher = more anomalous).
- **Autoencoder (detector)**: an MLP $48 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 48$ with ReLU activations, trained with Adam to minimise reconstruction MSE on normal training blocks, with early stopping on a held-out slice of the normal training data (it converged in ~ 19 epochs). Inputs are $\log(1 + \text{counts})$ then standardised, with the scaler fit on normal training data only. The per-block reconstruction MSE is the anomaly score. Both scores feed the *same* evaluation code, so the comparison is exactly apples-to-apples.

7.4 Results

Table 3: HDFS detection on the test split (172,518 blocks, 2.16% anomalous).

Metric	Isolation Forest	Autoencoder
PR-AUC (average precision)	0.748	0.987
Precision @ 90% recall	0.556	0.961
False alarms / day @ 90% recall	$\sim 12,700$	~ 650

Reading the numbers. The autoencoder’s PR-AUC of 0.987 (against a chance line of 0.022) means it ranks almost all anomalies above almost all normal blocks. Operationally it is decisive: at the same 90% detection rate it produces ~ 650 false alarms/day versus the baseline’s $\sim 12,700$ —a $\sim 20\times$ reduction. Figure 2 shows the full precision–recall curves; the autoencoder holds near-perfect precision until ~ 0.95 recall.

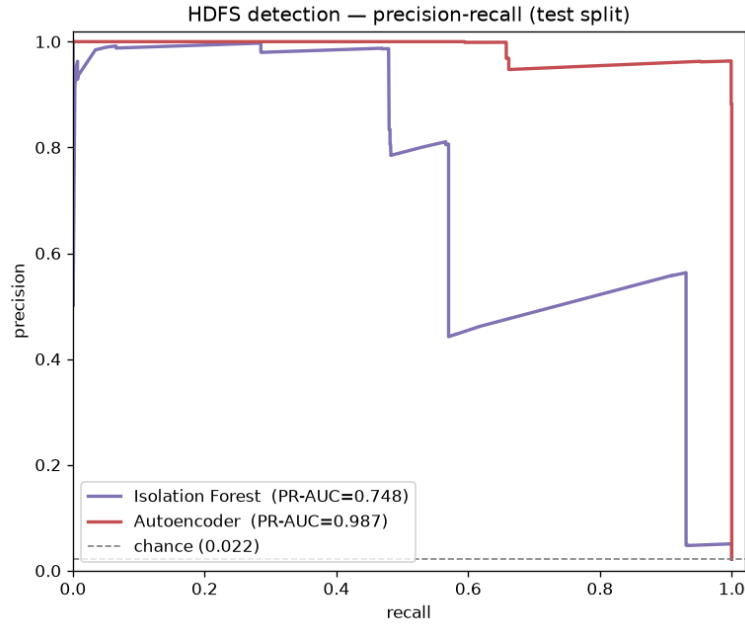


Figure 2: HDFS precision–recall, test split. The autoencoder (PR-AUC 0.987) dominates the Isolation Forest baseline (0.748) at every operating point; the dashed line is the 0.022 chance level.

The drift result (why this matters more than the headline). We also computed PR-AUC over four consecutive slices of the 5-hour test window. The baseline’s discrimination swung wildly (0.69–0.98), while the autoencoder held ≥ 0.99 throughout. So the autoencoder is not just more accurate but more *stable*—a property invisible to a single aggregate number and exactly what the drift check is designed to expose. Crucially, however, the *false-alarm rate* at a fixed threshold was *not* stable even for the autoencoder (it swung by orders of magnitude across the window)—the problem Section 10 then solves.

Key takeaway

The HDFS autoencoder is a strong *reproduction* (PR-AUC 0.987), used here to validate the pipeline and feed the triage layer—not claimed as novel. Its real lesson is the drift analysis: aggregates hide instability that only per-period measurement reveals.

8 The mathematical core: Hawkes process on BGL

This is the intellectual centerpiece. We model *when* BGL events happen with a self-exciting point process, estimate how cascade-prone the system is, validate the fit rigorously, and later test whether the model helps detection.

8.1 The model, restated with meaning

The exponential-kernel Hawkes intensity (Equation 1) is

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}.$$

Reading each symbol: μ is the rate of “spontaneous” events; α is how much the intensity jumps the instant an event occurs; β is the decay rate, so $1/\beta$ is the characteristic time over which one event’s influence fades. The branching ratio $\eta = \alpha/\beta$ (Section 2.6) is the mean number of direct offspring per event.

8.2 Estimating (μ, α, β) by maximum likelihood

We choose the parameters that make the observed event times most probable. For a point process observed on $[0, T]$, the log-likelihood is

$$\ell(\mu, \alpha, \beta) = \sum_i \log \lambda(t_i) - \int_0^T \lambda(s) ds. \quad (3)$$

The two terms have clean meanings. The first *rewards* placing high intensity exactly where events actually occurred (a good model is “surprised” little by the real events). The second *penalises* predicting a high total number of events overall (the integral is the model’s expected event count); it stops the trivial cheat of setting the intensity huge everywhere.

For the exponential kernel both terms are cheap. The integral is closed-form,

$$\int_0^T \lambda(s) ds = \mu T + \frac{\alpha}{\beta} \sum_i \left(1 - e^{-\beta(T-t_i)}\right), \quad (4)$$

and the sum $\sum_i \log \lambda(t_i)$ is computed in *linear* time using Ogata’s recursion: the excitation felt at event i , $A_i = \sum_{j < i} e^{-\beta(t_i - t_j)}$, satisfies

$$A_i = e^{-\beta(t_i - t_{i-1})}(1 + A_{i-1}), \quad A_1 = 0, \quad \lambda(t_i) = \mu + \alpha A_i. \quad (5)$$

This recursion is the reason the fit is feasible on millions of events—the naive double sum would be quadratic. We maximise ℓ over $(\log \mu, \log \alpha, \log \beta)$ (the log-parametrisation makes the positivity constraints automatic and the optimisation unconstrained) with the L-BFGS-B optimiser.

8.3 Correctness before trust: recovery from simulation

A fitted number is worthless unless the fitter is correct. Because this is the centerpiece, we *proved* the implementation before touching BGL: we simulate a Hawkes process with *known* parameters using Ogata’s thinning algorithm, then fit it and check the estimates match. They do (within a few percent). We also verified that the time-rescaling test *passes* a correct fit and *rejects* a deliberately-wrong model. These checks live in `tests/test_hawkes.py` and run in the automated test suite. We implemented everything in `scipy` because the standard `tick` library has no installable build for modern Python; the mathematics is library-independent.

8.4 Validating the fit with the time-rescaling theorem

As explained in Section 2.7, if the fit is good the rescaled inter-event gaps $\Delta\tau_i = \Lambda(t_i) - \Lambda(t_{i-1})$ are i.i.d. $\text{Exp}(1)$. We compute them, plot them against the ideal $\text{Exp}(1)$ quantiles (a QQ-plot: points on the diagonal = perfect fit), and run a KS test. A single interpretable diagnostic—the *mean* rescaled gap—should be exactly 1 if the compensator is correct, independent of shape; we use it as a normalisation sanity check.

8.5 Results and interpretation

Table 4: Hawkes fits on two BGL windows (all events).

BGL window	Events	$\eta = \alpha/\beta$	KS statistic	mean rescaled gap
2005-09-03 (calm day)	6,858	0.971	0.095	1.0001
2005-06-11 (fault cascade)	152,669	0.9998	0.431	1.0000

The headline: near-critical self-excitation. On both windows the branching ratio is $\eta \approx 0.97$ – 1.0 —remarkably close to the critical value 1. In plain terms, *each BGL event triggers on*

the order of one direct offspring event: the system sits right at the edge where cascades become self-sustaining. This is a precise, estimated, interpretable statement about the data, far stronger than “the logs are bursty.” Figure 3 (top) visualises the fitted intensity $\lambda(t)$: near-zero between bursts, spiking sharply (to ~ 190 events/s) as each cascade self-excites.

The honest caveat: where the single-exponential kernel breaks. The QQ-plots (Figure 3, bottom) show the *bulk* of the rescaled gaps lying on the Exp(1) diagonal—the model captures ordinary clustering well—but the *upper tail* bending upward: the longest quiet gaps are longer than a single exponential predicts. The mechanism is intuitive: right after a burst the fitted excitation expects many events; when silence follows instead, the model has “over-predicted,” and the integrated intensity over that gap is large, producing an outlying rescaled gap. A single exponential cannot simultaneously fit tight bursts and deep lulls. The effect is mild on the calm day (KS 0.095) and severe on the extreme cascade (KS 0.431, with η pinned at the critical boundary). The mean rescaled gap of exactly 1.0000 confirms the compensator is correctly normalised even where the shape deviates; and because the KS test has enormous power at 10^5 points (it rejects essentially any parametric model at that sample size), we judge the fit by the *shape* of the QQ-plot and the *magnitude* of deviation, not by the *p*-value.

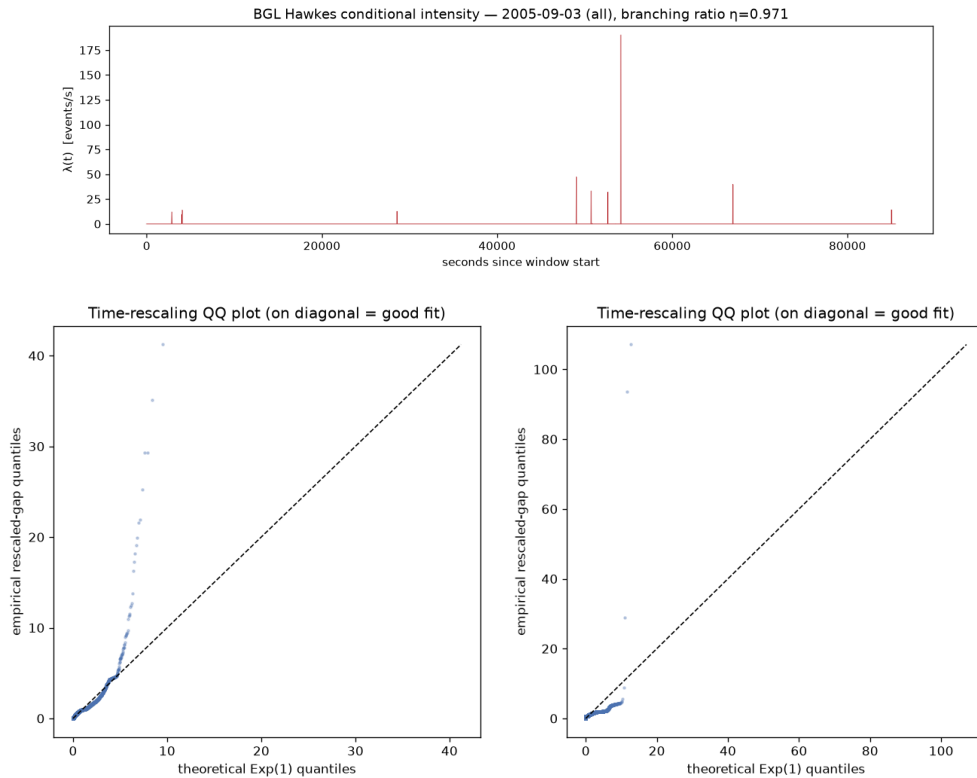


Figure 3: **Top:** fitted conditional intensity $\lambda(t)$ on the calm day —near-zero baseline with sharp self-exciting spikes. **Bottom:** time-rescaling QQ-plots for the calm window (left, KS 0.095) and the extreme cascade (right, KS 0.431). Both show the body on the ideal diagonal and the upper tail deviating—the single-exponential kernel underfits the deepest lulls.

Key takeaway

BGL’s event stream is *near-critically self-exciting* ($\eta \approx 0.97\text{--}1.0$): a concrete, validated finding. The single-exponential Hawkes captures the bulk clustering but underfits the deepest inter-event gaps—honestly characterised via time-rescaling, and motivating a richer kernel as future work.

9 Does the timing signal actually help detection?

Fitting a Hawkes process is elegant, but a professional reader will ask: does it *help*? We tested this directly rather than assuming, and report the honest answer. On fixed 60-second BGL windows we detect alert-containing windows three ways with the *same* model (Isolation Forest) and the *same* temporal split, changing *only* the features:

- **content** — counts of the top-50 event templates in the window (which events occurred; ignores timing).
- **timing** — the per-window Hawkes self-excitation $A(t) = \sum_{t_i < t} e^{-\beta(t-t_i)}$ summarised (mean/-max) at two decay scales (1s and 30s), plus the event count. Computed from event *times only*—no labels—so it is a legitimate, leakage-free detector feature. High $A(t)$ means “this window sits inside a self-exciting cascade.”
- **content+timing** — both feature blocks concatenated.

Table 5: BGL window detection, feature ablation (test split, 4.71% anomalous).

Feature set	PR-AUC	Precision @ 90% recall	False alarms / day
content	0.081	0.046	114
timing	0.087	0.083	61
content+timing	0.090	0.071	72
chance	0.047	—	—

The honest, nuanced answer. Two things are true at once. First, *absolute detection here is weak*: all three feature sets sit around PR-AUC 0.09, only $\sim 2\times$ the 0.047 chance line. Detecting BGL fault windows at 60-second resolution is genuinely hard—most alert windows fold a handful of alert lines into heavy normal traffic, so the unsupervised signal is faint. We do not oversell it. Second, and more interestingly, *the timing signal is the more useful one and it cuts false alarms*: at 90% recall, timing roughly *halves* the false-alarm rate versus content ($114 \rightarrow 61$ per day) and nearly doubles precision ($0.046 \rightarrow 0.083$). Figure 4 shows why: the content curve collapses to the chance line at high recall, while the timing curve holds $\sim 2\times$ chance across the operational range.

Key takeaway

On BGL the Hawkes self-excitation feature is the single most useful signal and roughly halves false alarms at a fixed detection rate—but it does not, alone, turn window-level detection into a strong detector. “Timing beats content and halves false alarms, on a hard task” is a real, validated result; a manufactured 0.98 would not be.

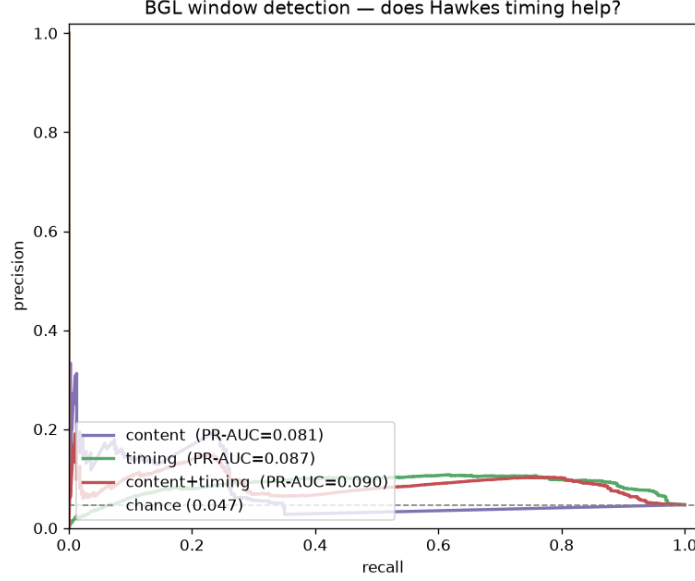


Figure 4: BGL window detection precision–recall. All feature sets are weak in absolute terms, but timing (green) and content+timing (red) clearly dominate content (purple), which collapses to the chance line at higher recall.

10 Adaptive thresholding: stabilising false alarms under drift

10.1 The problem, made concrete

A detector produces a continuous score; a *threshold* turns it into alerts. The drift analysis (Section 7) exposed the real operational weakness: at a *fixed* threshold, the false-alarm rate wanders as the score distribution drifts, even while ranking quality (PR-AUC) stays high. Measured on the HDFS autoencoder scores across the 5-hour test window, a fixed threshold’s false alarms swing from 3,664 to **861,686** per day—at the worst moments it flags essentially every block (Figure 5). No analyst can work with a false-alarm rate that varies by two orders of magnitude.

10.2 Extreme-Value Theory: the principled tool (derivation)

We want a threshold z_q exceeded by only a small target fraction q of scores (e.g. $q = 0.03$), *recomputed as the distribution drifts*. EVT (Section 2.8) says: pick a high pilot threshold t (e.g. the 95th percentile); the exceedances $X - t$ above it follow a Generalized Pareto Distribution with shape γ and scale σ ,

$$\Pr(X - t > x \mid X > t) \approx \left(1 + \frac{\gamma x}{\sigma}\right)^{-1/\gamma}. \quad (6)$$

Fit (γ, σ) to the observed exceedances (we use a fast method-of-moments estimator so it can update in $O(1)$ per new point). Then set the total tail probability equal to the target, $\Pr(X > z_q) = (N_t/n)(1 + \gamma(z_q - t)/\sigma)^{-1/\gamma} = q$, and solve for the threshold:

$$z_q = t + \frac{\sigma}{\gamma} \left[\left(\frac{qn}{N_t} \right)^{-\gamma} - 1 \right], \quad (7)$$

with n the sample size and N_t the number of exceedances above t . Holding q fixed while re-estimating (t, γ, σ) on a trailing window keeps the flagged fraction—hence the false-alarm rate—steady as the scores drift. We implemented this (**AdaptivePot**) and verified on simulated data that it controls the exceedance rate and tracks drift where a fixed threshold fails.

10.3 An honest negative finding, and the fix

When we applied EVT to the HDFS autoencoder scores it *misbehaved*: the fitted threshold sometimes fell below the data floor and flagged everything. The cause is real and instructive. The autoencoder scores are strongly *discrete*: 58.7% of normal blocks reconstruct to the *identical* value, because normal HDFS blocks are so repetitive that huge groups have byte-identical count vectors. GPD assumes a *continuous* tail; a distribution with a giant point mass violates that assumption, and the extrapolation breaks. So we did the honest thing: on HDFS we use the distribution-free **rolling quantile** (the empirical $(1 - q)$ -quantile of a trailing window, which makes no distributional assumption), and we reserve EVT for BGL’s genuinely continuous timing scores, where its extrapolation to small q earns its place.

10.4 Result

Table 6: HDFS false-alarm stability under drift (autoencoder scores, target flag rate $q = 3\%$).

	Fixed threshold	Adaptive (rolling quantile)
False alarms / day (range over test)	3,664–861,686	0–7,584
Flagged fraction	up to 100%	stable $\approx 2\%$
Recall	1.00	0.72

The adaptive threshold rises and falls to track the drifting scores, holding the alert rate near target instead of exploding (Figure 5). The transparent cost is recall: in periods where anomalies surge past the flag budget, holding the false-alarm rate constant means catching fewer of them ($1.00 \rightarrow 0.72$). This is the true operating-point tradeoff, now made *explicit and tunable* via the single knob q , rather than hidden.

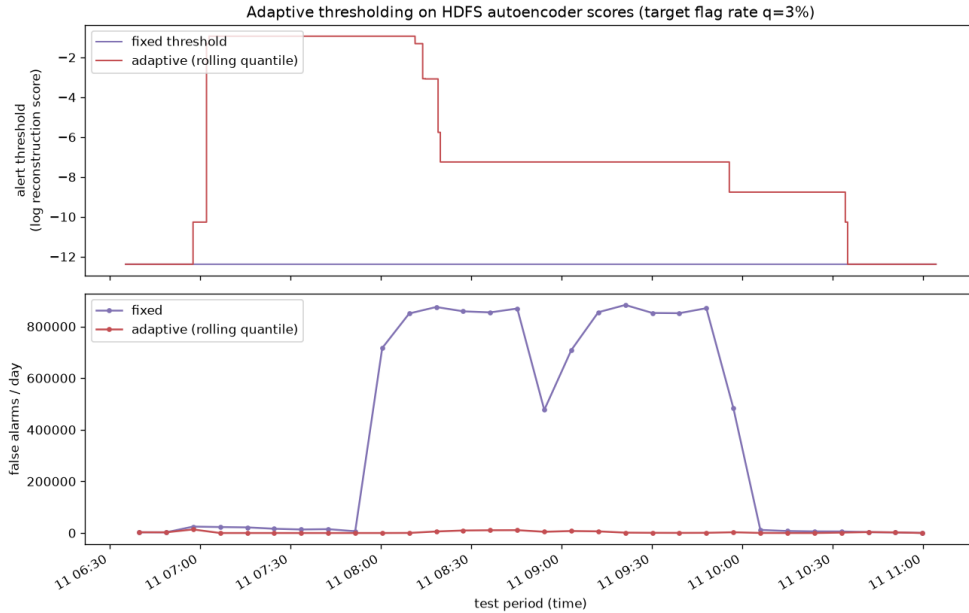


Figure 5: **Top:** the fixed threshold stays put while the normal-block scores drift upward mid-period; the adaptive threshold rises to track them. **Bottom:** the fixed threshold’s false alarms explode to $\sim 860\text{k/day}$ mid-period; the adaptive threshold holds the alert rate near target throughout.

Key takeaway

A fixed threshold is undeployable under drift—the false-alarm rate is what moves, and it moves by $\sim 235\times$. A trailing-window adaptive threshold stabilises it at a transparent, tunable recall cost. EVT is the right tool for continuous scores; HDFS’s discrete scores require the distribution-free rolling quantile instead—a finding we report rather than paper over.

11 LLM triage: from raw alerts to ranked incidents

Detection produces many flagged units; paging an analyst once per flag is unusable. The triage agent reduces volume, then explains, in three steps.

1. **Deduplicate.** Flags with the same *signature*—same entity (block/node) and same multiset of event templates—collapse into a single representative, carrying a count. Identical repeated alerts are recorded once, not re-paged.
2. **Correlate into incidents.** Flags close together in time are grouped into one *incident*. This is the Hawkes idea made operational: self-exciting events are causally linked, so a temporal cascade of anomalies is *one* root-cause incident, not two hundred separate pages. This grounding in the timing model is what distinguishes it from an arbitrary fixed-window heuristic.
3. **Explain and rank.** Each incident is classified into strictly-typed JSON—`priority`, `category`, `summary`, `recommended_action`, `explanation`—and the incidents are ranked P1→P4 by priority then peak score. The explainer is *pluggable*: a deterministic rule-based backend (category from template keywords; priority from severity and cascade size) runs fully self-contained, while an **Anthropic backend** (`claude-opus-4-8`, using *structured outputs* so the model is forced to return exactly the required JSON schema) drops in when an API key is configured, and falls back automatically if not. There is one model call per *incident*, not per raw flag—the dedup/correlation makes the LLM cost scale with incidents, not alerts.

Result. On the top-200 flagged HDFS blocks, the agent produced **200 raw flags** → **90 incidents**, ranked P1(13) / P2(53) / P3(2) / P4(22) across *data-loss*, *replication-failure*, and *network* categories (Figure 6). An operator sees 90 prioritised, explained incidents instead of 200 undifferentiated alerts.

Key takeaway

Triage turns a flood of flags into a short, ranked, explained incident list. Its one conceptual novelty is that correlation is grounded in the same self-excitation model as the rest of the project: a cascade is one incident.

Honest note on the LLM backend. This build environment has no model credential, so the demonstration ran on the deterministic rule-based explainer. The Anthropic path is written against the current API (structured outputs, `claude-opus-4-8`) and activates with a key; it was not exercised live here.

12 The dashboard and software architecture

Dashboard. A Streamlit application (`app/dashboard.py`) presents the whole pipeline in four tabs—Detection (metrics + PR curve), Thresholding (the drift-stabilisation plot), Hawkes (intensity + QQ), and Incidents (the ranked table with per-incident drill-down showing summary, action, and explanation). It was verified to start and render headless (Figure 6); “logs in, ranked and explained incidents out.”

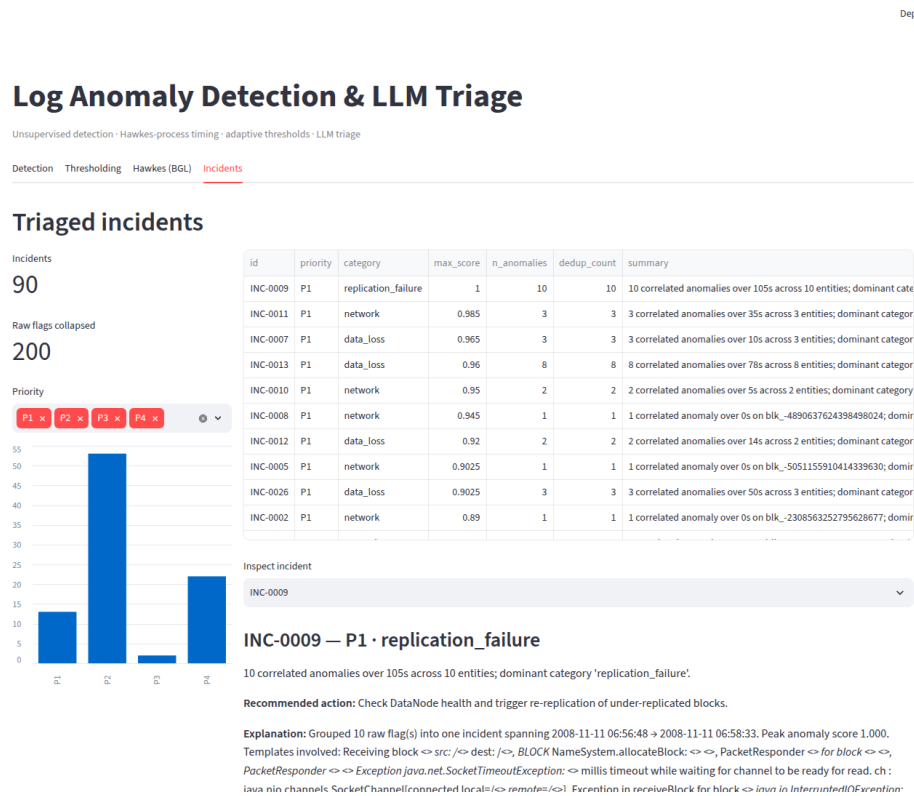


Figure 6: The dashboard’s Incidents tab: 200 raw flags collapsed to 90 ranked incidents, with the priority distribution and a per-incident drill-down.

Software architecture. The code is a small installable Python package, `src/logtrriage/`, with one sub-module per pipeline stage—**data** (download/load), **parsing** (Drain wrapper), **features** (block-session and window features), **models** (Isolation Forest, autoencoder), **hawkes** (intensity, MLE, simulation, time-rescaling), **eval** (PR-AUC, false-alarms- per-day, drift, EVT), and **trriage** (incident model, explainers, agent). Thin scripts in `scripts/` are the runnable entry points, one per stage, each reading the previous stage’s output from `data/`. This keeps logic testable and reused identically by scripts, notebooks, and the dashboard. **34 unit tests** cover the load-bearing logic—parameter recovery for the Hawkes fitter, the metric definitions, the EVT rate control, the triage correlation, and more—so a regression is caught immediately.

13 Results at a glance

Table 7: Consolidated results.

Result	Value
HDFS parsed	11.2M lines → 48 templates
BGL parsed	4.71M lines → 429 templates
Autoencoder PR-AUC (test)	0.987 (baseline 0.748)
False alarms/day @ 90% recall	~650 (baseline ~12,700)
Hawkes branching ratio (BGL)	$\eta \approx 0.97\text{--}1.0$ (near-critical)
Timing feature effect (BGL)	halves false alarms vs. content
Fixed → adaptive threshold	FA/day swing $\times 235$ → held near target
Triage reduction	200 flags → 90 ranked incidents
Unit tests	34 passing

14 Honest limitations

- **HDFS detection is a reproduction**, not a contribution; its role is validation and substrate, stated plainly.
- **BGL window detection is weak in absolute terms** (~ 0.09 PR-AUC); the timing result is a *relative* win (fewer false alarms) on a hard task, not a strong detector.
- **The single-exponential Hawkes kernel underfits** BGL’s deepest lulls (the time-rescaling upper tail); a sum-of-exponentials or power-law kernel is the natural fix.
- **EVT does not fit the discrete HDFS scores**; the rolling quantile is used there, and EVT is reserved for continuous scores.
- **The LLM triage backend was not exercised live** (no credential in this environment); only the rule-based path ran end-to-end.
- **Hyperparameters are first choices, not swept**: window size (60s), the top-50 template vocabulary, the two decay scales, and the network architecture were reasonable defaults, not tuned on a validation set.

15 Conclusion and future work

We built a complete, unsupervised log-anomaly pipeline with a stochastic-process core and an LLM triage layer, validated with a discipline—temporal split, PR-AUC, false-alarms-per-day, and drift—that most public reproductions skip. The mathematical centerpiece, a Hawkes process fit and validated on BGL, yields a concrete interpretable finding (a near-critical branching ratio) and an honest account of where a single-exponential kernel breaks. The timing signal it produces measurably reduces false alarms even though absolute detection on BGL is hard. Throughout, we separated what the community has already established from what we contributed, and treated negative and nuanced results as first-class outcomes.

Future work. (1) A richer Hawkes kernel (sum-of-exponentials / power-law) to fix the upper-tail misfit revealed by time-rescaling. (2) Use the time-rescaling *residual* directly as a calibrated timing-anomaly detector, with EVT thresholding on those continuous scores. (3) Sweep window size and decay scales, and try the autoencoder in place of Isolation Forest for the timing ablation. (4) Exercise the LLM triage end-to-end with a live model and evaluate incident quality against held-out labels.

A Notation

Symbol	Meaning
$\lambda(t)$	conditional intensity: instantaneous event rate given the past
μ	Hawkes background (spontaneous) rate
α	Hawkes excitation jump per event
β	Hawkes decay rate ($1/\beta$ = influence timescale)
$\eta = \alpha/\beta$	branching ratio: mean direct offspring per event
$\Lambda(t) = \int_0^t \lambda$	compensator (expected event count by time t)
$\Delta\tau_i$	rescaled inter-event gap ($\text{Exp}(1)$ under a correct fit)
q	target tail / flagged probability for the alert threshold
(γ, σ)	GPD shape and scale parameters (EVT tail)
TP,FP,FN,TN	true/false positive/negative counts

B Glossary

Log template / event type

The fixed “shape” of a family of log lines with variable parts blanked out; produced by Drain.

Block session (HDFS)

All log lines belonging to one HDFS block; the unit that carries a label and is scored.

Count vector

A window/block represented by how many times each template occurred—the detector’s input.

Autoencoder

A neural network trained to reconstruct its input through a bottleneck; reconstruction error is the anomaly score.

Isolation Forest

An unsupervised detector that scores points by how few random splits isolate them.

Precision / Recall

Fraction of alerts that are real / fraction of real anomalies caught.

PR-AUC

Area under the precision–recall curve; the imbalance-robust ranking metric.

Temporal split

Train on the earlier stream, test on the strictly later stream, never shuffled.

Point process

A random model of when events occur, described by its intensity $\lambda(t)$.

Hawkes process

A self-exciting point process: each event raises the intensity, decaying over time.

Branching ratio

$\eta = \alpha/\beta$; mean direct offspring per event; < 1 stable, > 1 explosive.

Time-rescaling theorem

A goodness-of-fit test: under a correct fit the rescaled gaps are $\text{Exp}(1)$.

EVT / POT / GPD

Extreme-Value Theory / Peaks-Over-Threshold / Generalized Pareto Distribution; the machinery for a self-calibrating tail threshold.

Drift

Change over time in the data or score distribution; measured per-period, not just in aggregate.

References

- [1] P. He, J. Zhu, Z. Zheng, M. R. Lyu. “Drain: An Online Log Parsing Approach with Fixed Depth Tree.” *IEEE ICWS*, 2017.
- [2] S. He, J. Zhu, P. He, M. R. Lyu. “Experience Report: System Log Analysis for Anomaly Detection.” *IEEE ISSRE*, 2016.
- [3] M. Du, F. Li, G. Zheng, V. Srikumar. “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning.” *ACM CCS*, 2017.
- [4] W. Meng et al. “LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs.” *IJCAI*, 2019.
- [5] A. G. Hawkes. “Spectra of Some Self-Exciting and Mutually Exciting Point Processes.” *Biometrika*, 1971.

- [6] Y. Ogata. “On Lewis’ Simulation Method for Point Processes.” *IEEE Trans. Information Theory*, 1981.
- [7] E. N. Brown, R. Barbieri, V. Ventura, R. E. Kass, L. M. Frank. “The Time-Rescaling Theorem and Its Application to Neural Spike Train Data Analysis.” *Neural Computation*, 2002.
- [8] A. Siffer, P.-A. Fouque, A. Termier, C. Largouët. “Anomaly Detection in Streams with Extreme Value Theory.” *ACM KDD*, 2017.
- [9] S. He, J. Zhu, P. He, M. R. Lyu. “Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics.” *IEEE ISSRE*, 2023.